tiny-os

Real-Time Operating System
for Raspberry Pi 5

Technical Specification

Target Platform: Broadcom BCM2712 / ARM Cortex-A76
Architecture: ARMv8.2-A (AArch64)

Version 1.1
March 2026
DRAFT

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Design Goals 4
1.4 Target Platform 4
1.4.1 BCM2712 Stepping Notes 5
1.5 Conformance 5
1.6 Terminology 5
2. System Architecture 7
2.1 Architectural Overview 7
2.2 Layer Model 7
2.3 Memory Map 7
2.4 Boot Sequence 8
2.5 Multicore Architecture 8
2.5.1 Single-Core Mode 8
2.5.2 SMP Mode 9
3. Task Management 10
3.1 Task Model 10
3.2 Task States 10
3.3 Task Control Block (TCB) 10
3.4 Task API 11
4. Scheduler 13
4.1 Scheduling Algorithm 13
4.2 Priority Scheme 13
4.3 Context Switch Mechanism 13
4.4 Time-Slicing 13
4.5 Scheduling Latency Targets 13
5. Inter-Process Communication 15
5.1 Semaphores 15
5.2 Mutexes 15
5.3 Message Queues 16
5.4 Event Flags 16
6. Memory Management 17

6.1 Design Philosophy 17
6.2 Memory Pools 17
6.3 Stack Overflow Detection 17
6.4 MMU Configuration 17
6.5 ASID Management 18
7. Timers and Timing Services 19
7.1 System Tick 19
7.2 Software Timers 19
7.3 High-Resolution Timing 19
8. Interrupt Management 19
8.1 GICv2 Architecture 19
8.2 Interrupt Priority Partitioning 20
8.3 Exception Vector Table 20
8.4 Deferred Interrupt Processing 20
8.5 Critical Sections 20
9. Device Driver Framework 22
9.1 Driver Model 22
9.2 Included Drivers 22
10. Configuration 23
10.1 Compile-Time Configuration 23
11. Error Handling 25
11.1 Error Codes 25
11.2 Hook Functions 25
12. Safety and Certification 26
12.1 Implementation Language 26
12.2 Testing 26
12.3 Certification Targets 26
13. Appendices 28
13.1 Appendix A: System Calls Summary 28
13.2 Appendix B: Bare-Metal Boot Configuration 28
13.2.1 config.txt 28
13.2.2 Build and Deploy 29
13.3 Appendix C: AArch64 Register Context 29
13.4 Appendix D: Revision History 29

1. Introduction

1.1 Purpose

tiny-os is a preemptive, priority-based real-time operating system designed to run bare-metal on the Raspberry Pi 5 single-board computer. It targets the Broadcom BCM2712 system-on-chip, which features a quad-core 64-bit ARM Cortex-A76 processor clocked at 2.4 GHz. tiny-os provides deterministic task scheduling, inter-task communication, memory management, and hardware abstraction while exploiting the performance and architectural features of the ARMv8.2-A instruction set.

1.2 Scope

This specification defines the architecture, interfaces, and behavior of tiny-os. It covers the kernel, scheduler, memory manager (including MMU configuration), inter-process communication facilities, device driver framework, multicore support, and system services. The specification is tailored to the BCM2712 SoC and the Raspberry Pi 5 board-level peripherals, including the RP1 southbridge I/O controller.

1.3 Design Goals

- Deterministic scheduling: worst-case interrupt-to-task latency under 1 microsecond on a single Cortex-A76 core at 2.4 GHz
- Minimal kernel footprint: core kernel image under 64 KB, kernel RAM usage under 16 KB (excluding application task stacks)
- Full AArch64 implementation: native 64-bit kernel exploiting ARMv8.2-A features including LSE atomics, cryptographic extensions, and the generic timer
- Configurable multicore support: run on a single core with the others parked, or enable SMP scheduling across all four Cortex-A76 cores
- MMU-based memory protection with per-task address space isolation using ARMv8-A translation tables
- Compile-time feature selection via Cargo features to include only required modules
- Support for both hard and soft real-time task classes

1.4 Target Platform

Attribute	Value
Board	Raspberry Pi 5 (all RAM variants: 1 GB, 2 GB, 4 GB, 8 GB, 16 GB)
SoC	Broadcom BCM2712 (C1 stepping on original 4/8 GB boards; D0 stepping on 1/2/16 GB and newer production runs of all variants)
CPU	Quad-core ARM Cortex-A76, 2.4 GHz, ARMv8.2-A (AArch64)
L1 Cache	64 KB I-cache + 64 KB D-cache per core
L2 Cache	512 KB per core
L3 Cache	2 MB shared
DRAM	LPDDR4X, 1 GB to 16 GB depending on variant
Interrupt Controller	ARM GICv2 (GIC-400)

Timer	ARMv8 Generic Timer (CNTPCT_EL0, CNTP_TVAL_EL0), 54 MHz reference clock
I/O Controller	RP1 southbridge via PCIe x4 (UART, SPI, I2C, GPIO, Ethernet, USB 3.0)
GPU	VideoCore VII (not used by tiny-os; reserved for GPU firmware and bootloader)
Boot	VideoCore GPU bootloader loads kernel8.img from FAT32 partition on SD card or NVMe via M.2 HAT+
Compatible Boards	Raspberry Pi 500 (keyboard-integrated, 8 GB, no external PCIe); Raspberry Pi 500+ (16 GB, mechanical keyboard, built-in M.2 SSD); Compute Module 5 (with CM5 IO board)

1.4.1 BCM2712 Stepping Notes

The BCM2712 D0 stepping, introduced with the 2 GB Raspberry Pi 5 in August 2024, removes unused dark silicon (features intended for other Broadcom markets such as set-top boxes) from the die. The D0 die is approximately 33% smaller than the C1, resulting in lower idle power consumption (approximately 30% reduction) and reduced manufacturing cost. From the perspective of tiny-os, the D0 stepping is functionally identical to the C1: the same Cortex-A76 quad-core CPU, the same GICv2, the same generic timer, and the same RP1 connectivity. tiny-os detects the stepping at boot by reading the MIDR_EL1 register and logs it to the debug UART, but no behavioral changes are required between steppings.

As of early 2026, Raspberry Pi is transitioning all variants (including 4 GB and 8 GB) to the D0 stepping as C1 stock is depleted. Both the 16 GB model (launched January 2025) and the 1 GB model (launched December 2025) shipped with D0 from initial production.

1.5 Conformance

tiny-os targets a subset of the POSIX Real-Time Extensions (IEEE 1003.1b) where applicable and provides a CMSIS-RTOS2-inspired API adapted for AArch64. The implementation language is Rust (no_std), with assembly for the exception vector table, context switch trampoline, and EL2-to-EL1 drop. Safety certification targets include IEC 61508 SIL-2 and ISO 26262 ASIL-B, leveraging the Ferrocene qualified Rust toolchain.

1.6 Terminology

Term	Definition
Task	An independent thread of execution with its own stack, register context, and priority level
TCB	Task Control Block; kernel data structure holding task state, priority, stack pointer, and MMU context
Tick	The fundamental time unit driven by the ARM Generic Timer, default 1 ms
EL0	Exception Level 0; unprivileged mode where application tasks execute

EL1	Exception Level 1; privileged mode where the tiny-os kernel executes
EL2	Exception Level 2; hypervisor level used only during boot to configure the system before dropping to EL1
GIC	Generic Interrupt Controller; ARM GICv2 (GIC-400) on the BCM2712
ISR	Interrupt Service Routine; handler dispatched via the GIC for hardware interrupts (IRQ/FIQ)
Critical Section	A code region where interrupts are masked or preemption is disabled to ensure atomicity
TTBR	Translation Table Base Register; holds the physical address of a task's page table root
WCET	Worst-Case Execution Time; maximum time a task or ISR can consume
IPI	Inter-Processor Interrupt; a software-generated interrupt sent from one core to another via the GIC

2. System Architecture

2.1 Architectural Overview

tiny-os follows a monolithic kernel architecture executing at EL1 on the ARM Cortex-A76. Application tasks run at EL0 with MMU-enforced memory isolation. The kernel is entered via synchronous exceptions (SVC instruction for system calls) and asynchronous exceptions (IRQ for hardware interrupts, including the generic timer tick and GIC-routed peripheral interrupts). All kernel objects are statically allocated at initialization to ensure deterministic behavior and avoid heap fragmentation.

2.2 Layer Model

Layer	Components	Exception Level
Application	User tasks, middleware, protocol stacks	EL0 (unprivileged)
System Call Interface	SVC dispatcher, argument validation, capability checks	EL1 (synchronous exception entry)
Kernel Services	IPC, timers, memory pools, task management	EL1 (privileged)
Scheduler	Per-core ready queues, context switcher, idle task, load balancer	EL1 (privileged)
HAL	GICv2, Generic Timer, MMU, UART, RP1 peripherals	EL1 (exception handlers)
Hardware	BCM2712 SoC, Cortex-A76 cores, LPDDR4X, RP1 southbridge	N/A

2.3 Memory Map

The BCM2712 on the Raspberry Pi 5 uses a 35-bit physical address space. The GPU firmware reserves the top portion of DRAM. tiny-os operates in the lower physical address range. The following table shows the physical memory layout for a 4 GB configuration (other variants scale the application region). On the 1 GB variant, the total available DRAM is approximately 948 MB after GPU reservation, requiring careful sizing of task stacks and memory pools via OS_CFG_MAX_TASKS and pool configuration.

Region	Physical Address Range	Size	Contents
Kernel Code	0x0008_0000 - 0x000F_FFFF	512 KB	Exception vectors, kernel text, read-only data
Kernel Data	0x0010_0000 - 0x001F_FFFF	1 MB	TCBs, ready queues, kernel state, page table pool
Task Stacks	0x0020_0000 - 0x00FF_FFFF	14 MB	Per-task kernel and user stacks
Memory Pools	0x0100_0000 - 0x01FF_FFFF	16 MB	Fixed-size block pools for IPC and application use
Application	0x0200_0000 - 0x3BFF_FFFF	~944 MB	Application code, data, and heap (MMU-mapped per task)

GPU Reserved	0x3C00_0000 - 0x3FFF_FFFF	64 MB	VideoCore VII firmware (do not access)
Peripherals (Legacy)	0xFE00_0000 - 0xFEFF_FFFF	16 MB	BCM2712 legacy peripheral registers
GIC-400	0xFF84_0000 - 0xFF84_FFFF	64 KB	GIC Distributor and CPU interface registers
RP1 (via PCIe)	0x1F_0000_0000 - 0x1F_0040_0000	4 MB	RP1 peripheral registers (UART, SPI, I2C, GPIO)

2.4 Boot Sequence

The Raspberry Pi 5 boot process is managed by the VideoCore GPU firmware, which loads the kernel image from the SD card (or NVMe SSD if configured for NVMe boot). tiny-os is provided as kernel8.img (a flat AArch64 binary) with an accompanying config.txt. The firmware's OS compatibility check must be disabled for bare-metal kernels by setting os_check=0 in config.txt. The boot sequence proceeds as follows:

1. GPU firmware powers on, initializes DRAM, reads config.txt and kernel8.img from the FAT32 boot partition
2. GPU loads kernel8.img at physical address 0x80000 and releases the primary CPU core (core 0) from reset at EL2
3. _start (assembly): saves the DTB pointer (x0), configures EL2 (HCR_EL2 for AArch64 EL1, timer access, no trapping of SIMD), then performs an ERET to drop to EL1
4. el1_entry (assembly): sets up the EL1 exception vector table (VBAR_EL1), initializes the kernel stack pointer (SP_EL1), zeroes BSS, and branches to kernel_main
5. kernel_main (Rust): parses the device tree blob to detect board revision, RAM size, and SoC stepping (C1 vs D0 via MIDR_EL1); initializes the GICv2 distributor and CPU interface; configures the MMU with an identity-mapped kernel page table; and enables the caches
6. Kernel initializes the TCB pool, per-core ready queues, timer subsystem, and memory pool allocator. Pool sizes are adjusted based on detected DRAM (critical for 1 GB variant)
7. If uart_early_init=1 was set in config.txt, the RP1 UART0 is already initialized by firmware at 115200 baud and the PCIe link to RP1 is preserved; otherwise, tiny-os must initialize the PCIe root complex and enumerate the RP1 before accessing any RP1 peripherals
8. Application registers initial tasks via os_task_create()
9. os_kernel_start() configures the generic timer for the tick interrupt, enables IRQs (DAIF clear), triggers the first context switch to the highest-priority ready task, and enters EL0
10. Secondary cores (1, 2, 3): if SMP is enabled, the primary core writes each secondary core's entry address to the spin table mailbox. Each secondary core initializes its own GIC CPU interface, per-core data structures, and enters the scheduler

2.5 Multicore Architecture

tiny-os supports both single-core and symmetric multiprocessing (SMP) configurations, controlled by the OS_CFG_SMP_EN compile-time flag.

2.5.1 Single-Core Mode

In single-core mode (default), tiny-os runs exclusively on core 0. Cores 1-3 remain in a WFE (Wait For Event) loop set up by the GPU firmware. This mode provides the simplest programming model and the most deterministic timing behavior, as there are no cache coherency or inter-core synchronization overheads.

2.5.2 SMP Mode

In SMP mode, tiny-os maintains a per-core ready queue and a global overflow queue. The scheduling model is as follows:

- Each core runs its own scheduler instance, selecting from its local ready queue
- Tasks are initially assigned to a core via an affinity mask set at creation time (default: any core)
- A global load balancer runs periodically (every 100 ticks by default) to redistribute tasks across cores when load imbalance exceeds a configurable threshold
- Inter-core communication uses IPIs (SGI 0-7) via the GIC to trigger reschedule events on remote cores
- Kernel data structures shared across cores are protected by ticket spinlocks with memory barriers (DMB ISH) to ensure cache coherency across the L1/L2 hierarchy
- Per-core data (current TCB, local ready queue, tick count) is accessed via TPIDR_EL1 to avoid shared-state contention

3. Task Management

3.1 Task Model

Each task in tiny-os is an independent thread of execution with a dedicated user-mode stack (EL0), a kernel-mode stack (EL1), a saved register context, and a dedicated translation table for memory isolation. Tasks are created statically at initialization. The maximum number of tasks is set at compile time via OS_CFG_MAX_TASKS (default: 32).

3.2 Task States

State	Description	Transitions To
READY	Task is eligible to run; in a core's ready queue or the global queue	RUNNING (selected by scheduler)
RUNNING	Task is currently executing on a CPU core at EL0	READY (preempted), BLOCKED (waiting), SUSPENDED, TERMINATED
BLOCKED	Task is waiting on a semaphore, mutex, queue, timer, or event flag	READY (event received or timeout)
SUSPENDED	Task is explicitly suspended via os_task_suspend()	READY (resumed via os_task_resume())
TERMINATED	Task has exited or been deleted; TCB and page table are reclaimable	N/A

3.3 Task Control Block (TCB)

The TCB is the core kernel data structure for each task. All TCBs are allocated from a static array during kernel initialization. The AArch64 TCB is larger than a typical Cortex-M TCB due to the wider register file, MMU context, and SMP affinity information.

Field	Type	Size	Description
sp_el0	u64	8 B	Saved user stack pointer (SP_EL0)
sp_el1	u64	8 B	Saved kernel stack pointer for this task
context	CpuContext	272 B	Saved X0-X30, SP_EL0, ELR_EL1, SPSR_EL1, Q0-Q31 (NEON/FP)
ttbr0	u64	8 B	TTBR0_EL1 value: physical address of this task's level-0 page table
asid	u16	2 B	Address Space Identifier for TLB tagging (avoids full TLB flush on switch)
stack_base_user	*mut u8	8 B	Base address of user-mode stack region
stack_size_user	u32	4 B	User stack size in bytes
stack_base_kernel	*mut u8	8 B	Base address of kernel-mode stack region

stack_size_kernel	u32	4 B	Kernel stack size in bytes
priority	u8	1 B	Base priority (0 = highest, 255 = lowest)
effective_priority	u8	1 B	Current effective priority (may differ due to priority inheritance)
state	OsState	1 B	Current task state (READY, RUNNING, BLOCKED, SUSPENDED, TERMINATED)
affinity_mask	u8	1 B	Bitmask of cores this task may run on (0x0F = any core)
current_core	u8	1 B	Core ID this task is currently assigned to
entry	fn(*mut c_void)	8 B	Pointer to task entry function
arg	*mut c_void	8 B	Argument passed to task entry function
name	&'static str	16 B	Human-readable task name (for debugging)
deadline	u64	8 B	Absolute tick count of next deadline (0 = no deadline)
delay_ticks	u64	8 B	Remaining ticks for os_delay() or timeout
blocked_on	Option<*mut OsObj>	8 B	Pointer to kernel object the task is waiting on
next	Option<*mut OsTcb>	8 B	Next pointer for linked list insertion (ready/wait queues)
stack_watermark	u32	4 B	Minimum observed free stack (for overflow detection)

3.4 Task API

The task API is exposed to application code via SVC system calls. The kernel validates all arguments at the EL1 entry point before performing the requested operation.

Function	Signature	Description
os_task_create	os_task_create(tcb: &mut OsTcb, entry: fn(*mut c_void), arg: *mut c_void, prio: u8, user_stack: &mut [u8], kern_stack: &mut [u8], name: &'static str, affinity: u8) -> OsErr	Create and register a task with core affinity; adds to ready queue
os_task_delete	os_task_delete(tcb: &mut OsTcb) -> OsErr	Terminate a task, release its TCB and page table
os_task_suspend	os_task_suspend(tcb: &mut OsTcb) -> OsErr	Move a task to SUSPENDED state
os_task_resume	os_task_resume(tcb: &mut OsTcb) -> OsErr	Move a SUSPENDED task back to READY
os_task_set_priority	os_task_set_priority(tcb: &mut OsTcb, prio: u8) -> OsErr	Change a task's base priority at runtime

os_task_set_affinity	os_task_set_affinity(tcb: &mut OsTcb, mask: u8) -> OsErr	Update the core affinity mask; may trigger migration
os_task_yield	os_task_yield()	Voluntarily relinquish the CPU to same-priority tasks
os_delay	os_delay(ticks: u64) -> OsErr	Block the calling task for a specified number of ticks
os_delay_until	os_delay_until(prev_wake: &mut u64, period: u64) -> OsErr	Block until an absolute tick count for periodic execution

4. Scheduler

4.1 Scheduling Algorithm

tiny-os uses a preemptive, fixed-priority scheduler with optional round-robin time-slicing among tasks of equal priority. In SMP mode, each core maintains an independent multi-level ready queue. The scheduler uses the CLZ (Count Leading Zeros) instruction on a 256-bit priority bitmap to locate the highest-priority non-empty queue in O(1) time.

4.2 Priority Scheme

- 256 priority levels: 0 (highest) through 255 (lowest)
- Priority 255 is reserved for the per-core idle task
- Priority 0 is reserved for the system watchdog task
- Application tasks should use priorities 1 through 254
- Multiple tasks may share a priority level; they are scheduled round-robin within that level

4.3 Context Switch Mechanism

Context switches on the Cortex-A76 are more involved than on Cortex-M due to the larger register file and the MMU. The switch is triggered by the kernel (after a timer tick, SVC, or IPC operation) and proceeds as follows:

1. The exception entry (IRQ or SVC) saves the full exception frame: ELR_EL1 (return address), SPSR_EL1 (saved PSTATE), and SP_EL0 onto the current task's kernel stack
2. The context save routine stores X0-X30 and Q0-Q31 (NEON/FP state, 32 x 128-bit registers) to the current TCB's CpuContext structure. Lazy FP save may be used if OS_CFG_LAZY_FP_EN is set, deferring Q-register save until the next task actually uses SIMD
3. The scheduler selects the next task from the local core's ready queue
4. If the next task has a different ASID/TTBR0, the kernel writes the new TTBR0_EL1 and issues a TLB invalidation by ASID (TLBI ASIDE1IS). The use of ASIDs (up to 256) avoids full TLB flushes in most cases
5. The context restore routine loads X0-X30 and Q0-Q31 from the new TCB's CpuContext
6. The exception return (ERET) atomically restores PSTATE from SPSR_EL1 and branches to the saved ELR_EL1, returning the new task to EL0 execution

4.4 Time-Slicing

When OS_CFG_TIMESLICE_EN is enabled, tasks at the same priority level are allocated a configurable time quantum (default: 10 ticks). When a task's quantum expires, the scheduler rotates it to the back of its priority queue and selects the next task at that level. Time-slice preemption does not apply to tasks with active deadline constraints.

4.5 Scheduling Latency Targets

Metric	Target (Cortex-A76 @ 2.4 GHz, single core)	Measurement Method
Interrupt latency (GIC to handler)	< 500 ns	Generic timer delta between GIC assertion and handler entry

Context switch (full, with MMU switch)	< 2 μs	PMU cycle counter between save-start and new-task-first-instruction
Context switch (same ASID, no MMU switch)	< 1 μs	PMU cycle counter
Scheduler decision	< 200 ns	CLZ-based O(1) lookup in priority bitmap
Tick jitter	< 100 ns	Generic timer comparator variance over 10,000 ticks
IPI latency (core-to-core)	< 1 μs	PMU cycle counter from SGI trigger to handler entry on target core

5. Inter-Process Communication

All IPC primitives in tiny-os are SMP-safe. In single-core mode, critical sections use interrupt masking. In SMP mode, IPC data structures are protected by short-hold spinlocks with interrupt masking on the local core to prevent deadlock.

5.1 Semaphores

tiny-os provides counting semaphores for synchronization and resource counting. Binary semaphores are a special case with a maximum count of 1.

Function	Signature	Description
os_sem_create	os_sem_create(sem: &mut OsSem, init: u32, max: u32) -> OsErr	Initialize a semaphore with initial and maximum count
os_sem_wait	os_sem_wait(sem: &mut OsSem, timeout: u64) -> OsErr	Decrement count; block if zero (timeout in ticks, 0 = forever)
os_sem_post	os_sem_post(sem: &mut OsSem) -> OsErr	Increment count; unblock highest-priority waiting task
os_sem_try_wait	os_sem_try_wait(sem: &mut OsSem) -> OsErr	Non-blocking attempt to decrement; returns OS_ERR_TIMEOUT if zero

5.2 Mutexes

Mutexes provide mutual exclusion with ownership tracking and priority inheritance to prevent priority inversion. Only the owning task may release a mutex.

Priority Inheritance: When a high-priority task blocks on a mutex held by a lower-priority task, the holder's effective priority is temporarily raised to the waiter's priority. This is transitive: if the holder is itself waiting on another mutex, the inheritance propagates. In SMP mode, the priority boost may trigger an IPI to the core running the holder, forcing an immediate reschedule so the holder can complete its critical section sooner.

Function	Signature	Description
os_mutex_create	os_mutex_create(mtx: &mut OsMutex) -> OsErr	Initialize a mutex in the unlocked state
os_mutex_lock	os_mutex_lock(mtx: &mut OsMutex, timeout: u64) -> OsErr	Acquire the mutex; block if held (with priority inheritance)
os_mutex_unlock	os_mutex_unlock(mtx: &mut OsMutex) -> OsErr	Release the mutex; restore priority; wake highest-priority waiter
os_mutex_try_lock	os_mutex_try_lock(mtx: &mut OsMutex) -> OsErr	Non-blocking acquisition attempt

5.3 Message Queues

Message queues allow tasks to exchange fixed-size messages through a FIFO buffer. Each queue is backed by a statically allocated memory region sized at compile time. In SMP mode, the queue's internal ring buffer is protected by a spinlock, and send/receive operations issue appropriate memory barriers to ensure cross-core visibility.

Function	Signature	Description
os_queue_create	os_queue_create(q: &mut OsQueue, buf: &mut [u8], msg_sz: u32, capacity: u32) -> OsErr	Initialize a queue with backing buffer
os_queue_send	os_queue_send(q: &mut OsQueue, msg: &[u8], timeout: u64) -> OsErr	Enqueue a message; block if full
os_queue_receive	os_queue_receive(q: &mut OsQueue, msg: &mut [u8], timeout: u64) -> OsErr	Dequeue a message; block if empty
os_queue_peek	os_queue_peek(q: &mut OsQueue, msg: &mut [u8]) -> OsErr	Read front message without removing it

5.4 Event Flags

Event flag groups allow a task to wait on one or more bits in a 32-bit flag word, supporting both AND (all specified bits set) and OR (any specified bit set) wait modes. Event flags use atomic operations (LDXR/STXR or LSE atomics when available) for lock-free set/clear in both task and ISR context.

6. Memory Management

6.1 Design Philosophy

tiny-os avoids dynamic heap allocation (malloc/free) in the kernel to prevent fragmentation and ensure deterministic allocation times. All kernel objects are statically allocated. For application use, tiny-os provides:

- Fixed-size memory pool allocators with O(1) allocation and deallocation for predictable-size objects
- MMU-based per-task memory isolation using ARMv8-A 4 KB granule translation tables
- Guard pages (unmapped) between task stacks for hardware-enforced stack overflow detection

6.2 Memory Pools

A memory pool is a pre-allocated region divided into equal-sized blocks managed via a free list. Allocation and deallocation are O(1) operations. In SMP mode, each pool has a spinlock; per-core pools are recommended for high-frequency allocation patterns.

Function	Signature	Description
os_pool_create	os_pool_create(pool: &mut OsPool, buf: &mut [u8], blk_sz: u32, blk_count: u32) -> OsErr	Initialize a pool from a static buffer
os_pool_alloc	os_pool_alloc(pool: &mut OsPool, timeout: u64) -> Result<*mut u8, OsErr>	Allocate a block; block if none available
os_pool_free	os_pool_free(pool: &mut OsPool, blk: *mut u8) -> OsErr	Return a block to the pool

6.3 Stack Overflow Detection

tiny-os uses the MMU for hardware-enforced stack overflow detection:

- Each task's user stack has a guard page (4 KB, unmapped) at its low end. A stack overflow triggers a Data Abort (translation fault) caught by the EL1 synchronous exception handler
- Each task's kernel stack similarly has a guard page. A kernel stack overflow triggers a fault that invokes os_hook_stack_overflow() with the faulting TCB
- Software watermark checking is also available: at each context switch, the scheduler inspects the stack watermark field and calls the hook if it falls below a configurable threshold

6.4 MMU Configuration

tiny-os configures the ARMv8-A MMU with a 4 KB translation granule and 48-bit virtual addresses (though only a fraction of the VA space is used). The translation table structure uses four levels (Level 0 through Level 3) for fine-grained mapping.

Kernel mapping (TTBR1_EL1): The kernel occupies the upper half of the virtual address space (addresses starting with 0xFFFF...). The kernel page table is shared across all tasks and

maps kernel code as read-only/execute, kernel data as read-write/no-execute, and device memory (peripherals, GIC, RP1) as Device-nGnRnE with no caching.

Task mapping (TTBR0_EL1): Each task has its own Level-0 page table referenced by TTBR0_EL1. Task memory is mapped in the lower half of the VA space. Each task's mapping includes its own code (read-only/execute), data (read-write/no-execute), stack (read-write/no-execute with guard pages), and any shared memory regions explicitly granted by the kernel.

The following MAIR (Memory Attribute Indirection Register) indices are used:

MAIR Index	Attribute	Usage
0	Device-nGnRnE	Peripheral registers (GIC, RP1, BCM2712 peripherals)
1	Normal, Inner/Outer Write-Back Cacheable, Read-Allocate/Write-Allocate	Kernel and task code/data in DRAM
2	Normal, Inner/Outer Non-Cacheable	DMA buffers, shared memory with non-coherent devices

6.5 ASID Management

tiny-os uses 8-bit ASIDs (256 values) to tag TLB entries per task. When a context switch occurs between tasks with different ASIDs, the kernel updates TTBR0_EL1 with the new physical page table address and ASID. The TLB retains entries from other ASIDs, avoiding a full flush. If the ASID space is exhausted (more than 255 active tasks, unlikely given OS_CFG_MAX_TASKS default of 32), a full TLB invalidation is performed and ASIDs are recycled.

7. Timers and Timing Services

7.1 System Tick

The system tick is driven by the ARMv8 Generic Timer, specifically the EL1 Physical Timer (CNTP_CTL_EL0, CNTP_TVAL_EL0, CNTP_CVAL_EL0). The generic timer runs at a fixed frequency independent of the CPU clock, providing a stable time base. On the BCM2712, the generic timer typically runs at 54 MHz. The default tick period is 1 ms (OS_CFG_TICK_RATE_HZ = 1000), configured by loading the appropriate comparator value.

In SMP mode, each core has its own generic timer instance, so tick interrupts are per-core and do not require IPI distribution.

7.2 Software Timers

tiny-os provides one-shot and periodic software timers that execute a callback function in the context of a dedicated timer service task (one per core in SMP mode). This avoids executing application code in ISR context.

Function	Signature	Description
os_timer_create	os_timer_create(tmr: &mut OsTimer, cb: fn(*mut c_void), arg: *mut c_void, period: u64, periodic: bool) -> OsErr	Create a software timer
os_timer_start	os_timer_start(tmr: &mut OsTimer) -> OsErr	Start or restart the timer
os_timer_stop	os_timer_stop(tmr: &mut OsTimer) -> OsErr	Stop a running timer
os_timer_delete	os_timer_delete(tmr: &mut OsTimer) -> OsErr	Remove a timer from the system

7.3 High-Resolution Timing

For sub-tick timing requirements, tiny-os provides access to the ARMv8 Performance Monitor Unit (PMU) cycle counter (PMCCNTR_EL0). The os_cycle_count() function returns the current CPU cycle count, enabling sub-nanosecond-precision profiling (at 2.4 GHz, one cycle is approximately 0.42 ns). The generic timer counter (CNTPCT_EL0) provides an absolute timestamp at the timer frequency (54 MHz, ~18.5 ns resolution) that is consistent across cores.

8. Interrupt Management

8.1 GICv2 Architecture

tiny-os uses the ARM GIC-400 (GICv2) on the BCM2712 for interrupt management. The GIC consists of two main components:

- Distributor (GICD): manages interrupt sources, priority levels, target core routing, and enable/disable state for all Shared Peripheral Interrupts (SPIs) and Private Peripheral Interrupts (PPIs)

- CPU Interface (GICC): per-core interface for interrupt acknowledgment, priority masking, and end-of-interrupt signaling. Each core has its own CPU interface instance

8.2 Interrupt Priority Partitioning

The GICv2 supports 256 priority levels (0 = highest). tiny-os partitions these into two bands:

Kernel-aware interrupts (priority >= OS_CFG_MAX_ISR_PRIO, default 0x40): These interrupts may call tiny-os API functions (os_sem_post, os_queue_send_from_isr, etc.). The kernel masks these during critical sections by writing OS_CFG_MAX_ISR_PRIO to GICC_PMR (Priority Mask Register).

Kernel-unaware interrupts (priority < OS_CFG_MAX_ISR_PRIO): These run above the kernel's priority mask, achieving near-zero-latency for ultra-time-critical peripherals such as motor controllers or high-speed ADC sampling. They must not call any tiny-os API functions.

8.3 Exception Vector Table

The AArch64 exception vector table (set via VBAR_EL1) contains 16 entries organized by exception type (synchronous, IRQ, FIQ, SError) and source (current EL with SP_EL0, current EL with SP_ELx, lower EL using AArch64, lower EL using AArch32). tiny-os uses the following vectors:

Vector	Source	Handler
Synchronous, Lower EL AArch64	SVC from EL0 task	svc_handler: dispatches system calls based on the SVC immediate value and X8 (syscall number)
IRQ, Lower EL AArch64	Hardware interrupt while in EL0	irq_handler: reads GICC_IAR, dispatches to registered handler, writes GICC_EOIR, checks for reschedule
IRQ, Current EL SP_ELx	Hardware interrupt while in EL1 (kernel)	irq_handler_el1: same dispatch but defers context switch until returning to EL0
Synchronous, Current EL SP_ELx	Data/instruction abort in EL1	fault_handler: kernel panic with register dump and stack trace
Synchronous, Lower EL AArch64 (Data Abort)	MMU fault from EL0 task	data_abort_handler: checks for guard page hit (stack overflow) or invalid access, kills task or invokes hook

8.4 Deferred Interrupt Processing

For ISRs that require significant processing, tiny-os supports the deferred interrupt pattern. The ISR performs minimal hardware servicing (acknowledge interrupt via GICC_IAR, read status registers, buffer data), then signals a high-priority handler task via a semaphore or event flag. The handler task runs at EL0 with full access to the kernel API and its own memory space.

8.5 Critical Sections

tiny-os provides the following mechanisms for critical sections:

- os_critical_enter() / os_critical_exit(): masks kernel-aware interrupts by writing OS_CFG_MAX_ISR_PRIO to GICC_PMR. Supports nesting via a per-core counter. Does not affect kernel-unaware (high-priority) interrupts
- os_irq_save() / os_irq_restore(): saves and restores the full interrupt state via DAIF register bits. Used when all interrupts must be masked, including kernel-unaware ones
- Spinlocks (SMP only): os_spin_lock() / os_spin_unlock() use ticket locks with LDAXR/STLXR (or LSE LDADD) for inter-core mutual exclusion, combined with local IRQ masking to prevent deadlock

9. Device Driver Framework

9.1 Driver Model

tiny-os defines a uniform driver interface for peripheral access. Drivers are implemented as Rust trait objects implementing the OsDriver trait. Each driver manages a specific peripheral and its associated interrupt(s).

On the Raspberry Pi 5, most I/O peripherals are behind the RP1 southbridge, which is connected to the BCM2712 via a PCIe 2.0 x4 link. The RP1 has its own set of peripheral registers mapped into the CPU's address space via BAR0 at physical address 0x1F_0000_0000 (mapping to RP1 internal address 0x4000_0000). Interrupts from RP1 peripherals are routed through the PCIe MSI mechanism to the GIC. For bare-metal operation, the recommended approach is to set pciex4_reset=0 and uart_early_init=1 in config.txt, which causes the GPU firmware to leave the PCIe link to RP1 configured and UART0 initialized. This avoids the need for tiny-os to implement a full PCIe root complex initialization sequence during early boot.

Trait Method	Signature	Description
init	fn init(&mut self, cfg: &dyn Any) -> OsErr	Initialize the peripheral with a configuration struct
open	fn open(&mut self, flags: u32) -> OsErr	Open a driver instance with access mode flags
close	fn close(&mut self) -> OsErr	Close the driver instance and release resources
read	fn read(&mut self, buf: &mut [u8], timeout: u64) -> Result<usize, OsErr>	Read data from the peripheral into a buffer
write	fn write(&mut self, buf: &[u8], timeout: u64) -> Result<usize, OsErr>	Write data from a buffer to the peripheral
ioctl	fn ioctl(&mut self, cmd: u32, arg: *mut c_void) -> OsErr	Perform device-specific control operations

9.2 Included Drivers

tiny-os ships with drivers for the Raspberry Pi 5 peripherals accessible via the RP1 southbridge and BCM2712 on-chip peripherals:

Driver	Peripheral	Bus	Features
os_drv_uart	PL011 UART (RP1)	RP1	Interrupt-driven TX/RX, configurable baud (up to 4 Mbps), hardware flow control
os_drv_spi	SPI (RP1)	RP1	Master mode, DMA-capable, configurable clock up to 50 MHz, multiple chip selects
os_drv_i2c	I2C (RP1)	RP1	Master mode, 100/400/1000 kHz, 10-bit addressing, timeout handling
os_drv_gpio	GPIO (RP1)	RP1	Pin configuration, interrupt-on-change (rising/falling/both), debounce, pull-up/down

os_drv_eth	Gigabit Ethernet (RP1)	RP1	Interrupt-driven MAC, scatter-gather DMA, MDIO for PHY management
os_drv_sd	SD/SDIO (Arasan)	BCM2712	SDR104 high-speed mode, interrupt-driven, DMA transfers
os_drv_pcie	PCIe 2.0 x1	BCM2712	Root complex management, BAR configuration, MSI interrupt routing
os_drv_mailbox	VideoCore Mailbox	BCM2712	Property tag interface for firmware queries (clock rates, board revision, MAC address)

10. Configuration

10.1 Compile-Time Configuration

tiny-os is configured via Cargo features and a central os_cfg module containing const values. All configuration constants use the OS_CFG_ prefix. The kernel statically allocates resources based on these settings, ensuring no runtime overhead for unused features.

Constant / Feature	Default	Description
OS_CFG_MAX_TASKS	32	Maximum number of tasks (determines TCB array size)
OS_CFG_TICK_RATE_HZ	1000	System tick frequency in Hz (1000 = 1 ms ticks)
OS_CFG_PRIO_LEVELS	256	Number of priority levels (8, 32, or 256)
OS_CFG_TIMESLICE_EN	true	Enable round-robin time-slicing at equal priorities
OS_CFG_TIMESLICE_TICKS	10	Time quantum for round-robin in ticks
OS_CFG_SMP_EN	false	Enable symmetric multiprocessing across all 4 cores
OS_CFG_SMP_CORES	4	Number of cores to activate when SMP is enabled
OS_CFG_MUTEX_EN (feature)	enabled	Include mutex module
OS_CFG_SEM_EN (feature)	enabled	Include semaphore module
OS_CFG_QUEUE_EN (feature)	enabled	Include message queue module
OS_CFG_TIMER_EN (feature)	enabled	Include software timer module
OS_CFG_EVENT_EN (feature)	enabled	Include event flag module
OS_CFG_STATS_EN (feature)	disabled	Enable CPU usage and stack statistics collection
OS_CFG_MAX_ISR_PRIO	0x40	GIC priority threshold for kernel-aware interrupts

OS_CFG_LAZY_FP_EN	true	Defer NEON/FP register save until actually needed
OS_CFG_IDLE_HOOK_EN	true	Call os_hook_idle() from the per-core idle task
OS_CFG_USER_STACK_SIZE	16384	Default user stack size per task in bytes (16 KB)
OS_CFG_KERN_STACK_SIZE	4096	Default kernel stack size per task in bytes (4 KB)
OS_CFG_LOAD_BALANCE_INTERVAL	100	Ticks between SMP load balancer runs
OS_CFG_MIN_DRAM_MB	64	Minimum DRAM in MB required for kernel boot (panic if less)
OS_CFG_AUTO_POOL_SIZING	true	Automatically scale memory pool sizes based on detected DRAM at boot

11. Error Handling

11.1 Error Codes

All tiny-os API functions return an OsErr enum. Application code should always check return values. The Rust type system enforces this via #[must_use] on OsErr.

Variant	Value	Meaning
OsErr::None	0	Operation completed successfully
OsErr::Timeout	1	Operation timed out before completion
OsErr::InvalidArg	2	A function argument is null, out of range, or violates a precondition
OsErr::InvalidState	3	Object is in an invalid state for the requested operation
OsErr::NoMemory	4	Memory pool exhausted; no blocks available
OsErr::Overflow	5	Semaphore count or queue capacity exceeded
OsErr::NotOwner	6	Caller is not the mutex owner
OsErr::IsrContext	7	API function called from ISR that requires task context
OsErr::KernelNotRunning	8	API called before os_kernel_start()
OsErr::AffinityViolation	9	Task cannot be migrated; no eligible core in affinity mask
OsErr::PermissionDenied	10	Task attempted an operation outside its granted capabilities

11.2 Hook Functions

tiny-os provides user-definable hook functions for critical system events. These are implemented as weak symbols (via #[linkage = "weak"]) that the application can override:

Hook	Trigger	Default Behavior
os_hook_idle(core: u8)	Called continuously from the per-core idle task	Execute WFI (Wait For Interrupt) for power savings
os_hook_stack_overflow(tcb: &OsTcb)	Guard page fault or watermark below threshold	Kernel panic with register dump and faulting task info
os_hook_data_abort(tcb: &OsTcb, addr: u64, esr: u64)	EL0 data abort (invalid memory access)	Terminate the faulting task; call os_task_delete()
os_hook_hard_fault(esr: u64, elr: u64, far: u64)	Unrecoverable EL1 exception	Kernel panic with full register dump to UART
os_hook_assert(file: &str, line: u32)	os_assert!() macro failure	Kernel panic with source location output to UART
os_hook_task_create(tcb: &OsTcb)	After a task is created	No-op

os_hook_task_switch(from: &OsTcb, to: &OsTcb, core: u8)	Before each context switch	No-op

12. Safety and Certification

12.1 Implementation Language

tiny-os is implemented in Rust targeting the aarch64-unknown-none target triple. The kernel uses #[no_std] and #[no_main] with no dependency on the Rust standard library or an allocator. Assembly is used only for the exception vector table, context switch trampoline, EL2-to-EL1 transition, and cache/TLB maintenance operations. All assembly is contained in .S files linked via global_asm!() or in dedicated naked functions.

The use of unsafe code is minimized and confined to well-documented modules: MMU page table manipulation, register access (MSR/MRS), context save/restore, and raw pointer operations in the memory pool allocator. Each unsafe block includes a SAFETY comment documenting the invariants that make the operation sound.

12.2 Testing

tiny-os includes a comprehensive test suite:

- Unit tests: each kernel module is tested using a QEMU virt machine target (aarch64) with GICv2 emulation, achieving over 95% branch coverage
- Integration tests: multi-task scenarios exercising IPC, preemption, priority inheritance, deadline handling, and SMP task migration, run on both QEMU and physical Raspberry Pi 5 hardware
- Stress tests: continuous operation under maximum load (32 tasks, all cores active, sustained IPC traffic) for 72+ hours with timing and memory validation
- Fault injection: systematic injection of stack overflows, invalid memory accesses, invalid syscall arguments, and ASID exhaustion to verify graceful error handling
- Miri and ASAN: unsafe code sections are exercised under Miri (where feasible) and address sanitization to detect undefined behavior

12.3 Certification Targets

tiny-os is designed to support certification under the following safety standards, leveraging the Ferrocene qualified Rust toolchain:

Standard	Level	Domain	Toolchain
IEC 61508	SIL-2	Industrial automation and functional safety	Ferrocene (qualified)
ISO 26262	ASIL-B	Automotive embedded systems	Ferrocene (ASIL-D qualified)
DO-178C	DAL-D (planned)	Avionics software (future release)	Ferrocene (in progress)
IEC 62304	Class B	Medical device software	Ferrocene (qualified)

13. Appendices

13.1 Appendix A: System Calls Summary

The following table summarizes all public API functions grouped by module. All functions are prefixed with os_ and return OsErr unless otherwise noted. System calls are dispatched via SVC with the syscall number in X8.

Module	Functions
Kernel	os_kernel_init, os_kernel_start, os_kernel_lock, os_kernel_unlock, os_kernel_get_tick, os_kernel_get_core_id
Task	os_task_create, os_task_delete, os_task_suspend, os_task_resume, os_task_set_priority, os_task_set_affinity, os_task_yield, os_delay, os_delay_until
Semaphore	os_sem_create, os_sem_wait, os_sem_post, os_sem_try_wait, os_sem_delete
Mutex	os_mutex_create, os_mutex_lock, os_mutex_unlock, os_mutex_try_lock, os_mutex_delete
Queue	os_queue_create, os_queue_send, os_queue_receive, os_queue_peek, os_queue_delete
Event Flags	os_event_create, os_event_set, os_event_clear, os_event_wait, os_event_delete
Timer	os_timer_create, os_timer_start, os_timer_stop, os_timer_delete
Memory Pool	os_pool_create, os_pool_alloc, os_pool_free
Driver	os_drv_register, os_drv_open, os_drv_close, os_drv_read, os_drv_write, os_drv_ioctl
Timing	os_cycle_count, os_timestamp

13.2 Appendix B: Bare-Metal Boot Configuration

To boot tiny-os on a Raspberry Pi 5, prepare an SD card with the standard Raspberry Pi firmware files and the following configuration:

13.2.1 config.txt

The following config.txt entries are required to boot a bare-metal AArch64 kernel on the Pi 5. These options reflect the official Raspberry Pi firmware documentation as of early 2026:

Entry	Value	Purpose
arm_64bit	1	Boot CPU cores in AArch64 mode
kernel	kernel8.img	Name of the tiny-os binary on the boot partition
os_check	0	Disable firmware OS compatibility check (required for bare-metal; without this, firmware rejects non-Linux kernels)

uart_early_init	1	Firmware initializes RP1 UART0 at 115200 baud and preserves the PCIe link to RP1 before starting the kernel (critical for early debug output on GPIO 14/15)
pciex4_reset	0	Disable PCIe x4 controller reset before kernel start; allows tiny-os to inherit the firmware's PCIe configuration for RP1 access without writing a full PCIe RC driver during early boot
enable_uart	1	Enable UART output on GPIO 14/15 (redundant if uart_early_init=1 but included for compatibility with older firmware)
dtoverlay	disable-bt	Disable Bluetooth to free PL011 UART for debug output
core_freq_min	1500	Set minimum core frequency to prevent DVFS during real-time operation (reduces timing jitter from frequency scaling)
arm_boost	1	Enable the 2.4 GHz turbo mode on Cortex-A76 cores (default on newer firmware)

13.2.2 Build and Deploy

The tiny-os kernel is built with the following commands:

1. Install the Rust aarch64-unknown-none target: rustup target add aarch64-unknown-none
2. Build: cargo build --release --target aarch64-unknown-none
3. Convert ELF to raw binary: aarch64-none-elf-objcopy -O binary target/aarch64-unknown-none/release/tiny-os kernel8.img
4. Copy kernel8.img to the FAT32 boot partition alongside config.txt, start4.elf, fixup4.dat, and bcm2712-rpi-5-b.dtb
5. Connect a USB-to-UART adapter to GPIO 14 (TX) and GPIO 15 (RX) at 115200 baud for debug output
6. Insert the SD card and power on the Raspberry Pi 5

13.3 Appendix C: AArch64 Register Context

The full context saved and restored on each context switch consists of:

- General-purpose registers: X0-X30 (31 x 64-bit = 248 bytes)
- Stack pointer: SP_EL0 (8 bytes)
- Exception return state: ELR_EL1, SPSR_EL1 (16 bytes)
- NEON/FP registers: Q0-Q31 (32 x 128-bit = 512 bytes), plus FPCR and FPSR (8 bytes). Saved lazily if OS_CFG_LAZY_FP_EN is enabled
- Total context size: 792 bytes (full) or 272 bytes (without NEON/FP)

13.4 Appendix D: Revision History

Version	Date	Author	Description

1.0	March 2026	tiny-os Team	Initial specification for Raspberry Pi 5 / Cortex-A76 (evolved from simple_os Cortex-M spec)
1.1	March 2026	tiny-os Team	Added BCM2712 D0 stepping documentation; added 1 GB RAM variant support and auto pool sizing; updated config.txt with official bare-metal options (os_check, uart_early_init, pciex4_reset); added compatible boards (Pi 500, Pi 500+, CM5); added stepping detection at boot; updated memory map notes for constrained-RAM variants